



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SMART PARKING MANAGEMENT AND PREDICTIVE SPACE ALLOCATION USING QUEUES AND HASH TABLES

A CAPSTONE PROJECT REPORT

A Capstone Project Report submitted in partial fulfilment of the requirements

for the Course of

CSA0311 – DATA STRUCTURES

to the award of the degree of

BACHELOR OF ENGINEERING

IN

INFORMATION TECHNOLOGY

Submitted by

R. RANJITHA (192421285)

Under the Supervision of

DR K. KIRUTHIKA

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026



SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105



BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “Smart Parking Management and Predictive Space Allocation Using Queue and Hash Table” has been carried out by R. Ranjitha (192421285) under the supervision of Dr. K. Kiruthika and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech Information Technology program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr K. Kiruthika,
Professor,
Department of CSE/ Programming,
SIMATS Engineering,
SIMATS, Chennai – 602105.

COURSE FACULTY

Dr K. Kiruthika,
Professor,
Department of CSE/Programming,
SIMATS Engineering,
SIMATS, Chennai – 602105. .



SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105



DECLARATION

I, R. Ranjitha (192421285) of the Information technology program, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “SMART PARKING MANAGEMENT AND PREDICTIVE SPACE ALLOCATION USING QUEUE AND HASH TABLE” is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, software resources, datasets, and external materials used in the project have been appropriately acknowledged. The Module 1 implementation documented in this report was developed as a C prototype intended to be compiled and executed using Visual Studio Code with a C-compatible compiler.

Place: Thandalam, Chennai

Date: September 2026

Signature of the Students:

R. Ranjitha (192421285)



SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai-602105



INDIVIDUAL CONTRIBUTION STATEMENT

Student	Specific Responsibilities	Design & Development	Testing & Analysis	Report Contribution	Approx. Contribution
R. Ranjitha (192421285)	Introduction, Module coordination, slot allocation logic, priority-queue implementation	Module 1 allocation and registration implementation	Functional testing and allocation validation	Introduction, Module 1, Results (portal)	35%
N. Umasri (192424240)	Vehicle data management and hash-based lookup	Vehicle registration and search implementation	Registration, duplicate and lookup testing	Objectives, Module 3, Future Scope	35%
B. Kiruthika (192511232)	Documentation, integration and presentation support	Integration and interface/testing support	System-level testing and documentation	Problem Statement, Module 2, Conclusion	30%
Total					100%

ABSTRACT

Urban centres and institutional campuses are experiencing increasing demand for parking facilities as vehicle ownership grows faster than available parking infrastructure. Traditional parking management approaches can require manual identification of free spaces, repeated vehicle searching, and separate maintenance of vehicle and slot records. These limitations can increase waiting time, internal congestion, and inefficient utilization of available parking capacity. This capstone project proposes a Smart Parking Management and Predictive Space Allocation System using advanced data structures. The overall project uses data structures including graphs, priority queues, hash tables, queues, and linked lists for parking operations, while the supplied project report additionally describes min-heaps, AVL trees, weighted graphs, and predictive scheduling. This report concentrates specifically on Module 1, Smart Parking Registration and Slot Allocation. The module provides vehicle registration, unique parking-ID generation, real-time slot-status management, automatic selection of a suitable available slot, and update of vehicle and parking records. For the VS Code implementation, the Module 1 prototype is developed in C17. An unordered map is used as the hash-table representation for rapid vehicle-number lookup, while a priority queue is used to select the nearest available parking slot based on distance from the entrance. A vector maintains the current slot records. When a vehicle is registered, the system generates a unique parking ID. During allocation, the system checks registration and availability, selects the highest-priority free slot, marks it Occupied, and stores the vehicle-slot relationship. When the vehicle exits, the assigned slot is changed back to Available. The implementation is designed as a functional academic prototype rather than a production deployment. The current Module 1 scope does not include physical occupancy sensors, database integration, payment processing, machine-learning forecasting, or the navigation and analytics modules. These are retained as future or separate modules. Testing focuses on registration, duplicate prevention, allocation, occupied-slot prevention, release, search, and no-availability conditions.

Keywords: Smart Parking, Slot Allocation, Hash Table, Priority Queue, Vehicle Registration, Data Structures

TABLE OF CONTENTS

SL. NO	TITLE	PAGE NO
i	CERTIFICATE FROM THE SUPERVISOR	
ii	DECLARATION BY THE CANDIDATE	III
lii	INDIVIDUAL CONTRIBUTION STATEMENT	IV
iv	ABSTRACT	V
v	LIST OF FIGURES	VII
vi	LIST OF TABLES	VIII
vii	LIST OF ABBREVIATIONS	IX
1	CHAPTER 1: Introduction	1
2	CHAPTER 2: Literature Review	5
3	CHAPTER 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security	8
4	CHAPTER 4: Implementation, Testing & Results, Project Management	17
5	CHAPTER 5: Conclusion & Reflection	25
	References	29
	Appendices	30

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	PAGE NO.
Figure 3.1	System Architecture	20
Figure 4.1	Vehicle Registration	25
Figure 4.2	Smart Slot Allocation	27
Figure 4.3	Slot Status and Release	27
Figure 5.1	Flow Chart of the Smart Parking Allocation	34
Figure 6.1	Review photo	38

LIST OF TABLES

TABLE NO.	TABLE NAME	PAGE NO.
Table 2.1	Comparative Analysis of Existing Approaches	15
Table 3.1	Stakeholder / User Requirements	16
Table 3.2	Functional & Non-Functional Requirements	16
Table 3.3	Constraints and Applicable Standards	18
Table 3.4	Design Specifications	19
Table 3.5	Alternative Solutions Evaluation Matrix	20
Table 3.6	Trade-off Analysis	22
Table 3.7	Sustainability, Ethics, Safety and Security Evaluation	23
Table 3.8	Risk Register	23
Table 4.1	Resources / Technologies Used	24
Table 4.2	Tools / Software Used and Implementation Stage	24
Table 4.3	Test Cases and Acceptance Criteria	27
Table 4.4	Specification Verification Results	27
Table 4.5	Achievement of Objectives	29
Table 4.6	Team Roles and Contributions	29
Table 4.7	Individual Contribution Percentage	31
Table 4.8	Project Timeline / Milestones	32
Table 4.9	Budget Estimation	35

LIST OF ABBREVIATIONS / SYMBOLS

Symbol / Term	Description
ID	Identification
API	Application Programming Interface
AVL	Adelson-Velsky and Landis self-balancing tree
IoT	Internet of Things
WCAG	Web Content Accessibility Guidelines
ISO/IEC 25010	Software product quality model
$O(1)$	Constant time complexity
$O(\log n)$	Logarithmic time complexity
C 17	C language standard used for the prototype
m	Distance from entrance

CHAPTER 1: INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

The supplied project material identifies inefficient parking management as an important practical problem caused by increasing vehicle demand, limited parking capacity, and the limitations of manual or basic occupancy-based systems. With the continuous increase in the number of vehicles in urban areas, educational institutions, commercial complexes, hospitals, and other public facilities, the demand for organized parking management has become increasingly important. Drivers may spend considerable time searching for a suitable parking space, while facility operators need to continuously maintain accurate information about occupied, available, and reserved spaces. Such difficulties can lead to unnecessary vehicle movement, internal congestion, inefficient utilization of parking capacity, and delays in parking operations. Traditional parking systems may depend on manual staff decisions or simple displays that only indicate whether spaces are available. Although these approaches can provide basic information, they do not always provide an efficient mechanism for automatically assigning the most suitable parking slot to a particular vehicle. As the number of vehicles and parking spaces increases, manually maintaining vehicle records and slot information can also become difficult and time-consuming. Therefore, an automated system based on appropriate data structures and algorithms can provide a more systematic and reliable solution. The original project concept addresses this problem by treating parking management as a data-structure and algorithm problem. Efficient data structures can be selected according to the operations that are performed most frequently. For example, vehicle records require fast searching using the vehicle number as a key, while parking-slot allocation requires the system to identify the most suitable free slot according to a defined priority. The broader system described in the supplied material uses graphs, priority queues, hash tables, queues, and linked lists, with another detailed project report specifying min-heaps for nearest-slot retrieval, hash maps for vehicle-to-slot lookup, AVL trees for zone availability, weighted graphs for navigation, and priority queues for predictive scheduling. These data structures support different operational requirements within the overall smart parking system. Module 1 forms the basic operational layer of this system. It creates the vehicle record, assigns a unique parking identifier, determines slot availability, selects a suitable slot, updates the corresponding records, and maintains current parking-slot status. In Module 1, the emphasis is placed on vehicle registration and intelligent slot allocation

using suitable data structures. A hash-table-based structure is used for efficient vehicle lookup, while a priority queue is used to organize available parking slots according to their priority. In the present prototype, the distance of a parking slot from the entrance is used as the main priority factor. This provides a clear and deterministic mechanism for selecting a suitable available slot and establishes a foundation for integrating additional smart parking features in later modules.

1.2 Need for the Project

- Traditional parking management may depend on manual staff decisions or basic availability displays, which may not be sufficient when the number of vehicles increases significantly.
- Drivers can spend additional time searching for a suitable free slot, resulting in unnecessary vehicle movement, increased waiting time, and congestion within the parking facility.
- Vehicle and slot records require fast retrieval when the facility becomes busy, particularly when attendants need to identify a registered vehicle or determine its assigned parking location.
- Available slots should be selected systematically rather than by arbitrary manual choice so that the allocation process remains consistent, predictable, and easy to manage.
- An automated allocation mechanism can reduce unnecessary searching and improve consistency by applying a predefined priority rule to the available parking spaces.
- Efficient data structures are required to support frequent operations such as vehicle registration, vehicle searching, slot allocation, slot-status updating, and slot release.
- A hash-table-based vehicle record structure can provide fast average lookup of registered vehicles, reducing the need for repeated sequential searching through all stored records.
- A priority queue can organize available parking slots according to their priority and provide an efficient mechanism for selecting the most suitable free slot.
- Automated slot-status management helps prevent an occupied parking space from being assigned to another vehicle and maintains consistency between vehicle and slot records.
- The system provides a structured foundation for demonstrating how data structures and algorithms can be applied to a real-world engineering problem involving resource allocation and record management.

- Module 1 provides the foundation required by later navigation, billing, analytics, sensor integration, and predictive functions. By establishing reliable registration and allocation operations first, the overall project can be extended with additional modules in a systematic manner.

1.3 Problem Statement

Traditional parking systems often lack an integrated mechanism that registers vehicles, generates unique parking identifiers, checks real-time slot status, and automatically assigns a suitable available slot using efficient data structures. In many conventional environments, vehicle registration and parking-slot allocation may be handled manually or through basic systems that provide limited information about parking availability. Such approaches can become inefficient when the number of vehicles, users, and parking spaces increases.

The proposed Module 1 should also generate a unique parking ID for each successfully registered vehicle and provide a mechanism for quickly retrieving vehicle information using the vehicle number. For parking allocation, the system should evaluate the available slots and select the highest-priority suitable slot according to the defined allocation rule. In the current prototype, the parking slot closest to the entrance is given higher priority than slots located farther away.

Therefore, Module 1 aims to provide a simple, efficient, and demonstrable software solution for smart parking registration and slot allocation using appropriate data structures. The implementation is intended as an academic prototype that demonstrates the practical application of hash tables, priority queues, and other supporting structures to solve parking-management problems. The module also establishes a technical foundation for the integration of advanced features such as sensor-based occupancy detection, predictive parking analysis, navigation, database persistence, billing, and web-based management in future stages of the project.

1.4 Project Aim

To design and implement an efficient Smart Parking Registration and Slot Allocation module that automates vehicle registration, unique parking-ID generation, real-time slot-status management, and intelligent selection of an available parking slot using suitable data structures.

1.5 Project Objectives

1. To design a data model for vehicles, parking IDs, and parking slots.
2. To implement vehicle registration and prevent duplicate vehicle records.
3. To generate a unique parking ID for every successfully registered vehicle.
4. To maintain Available, Occupied, and Reserved slot states.
5. To automatically select the most suitable available slot using a priority-based structure.
6. To provide fast vehicle search through hash-based indexing.
7. To update slot status consistently after allocation and release.
8. To test the module under normal, duplicate, occupied, and no-availability conditions.

1.6 Scope

Included

- Registration and storage of vehicle details, including vehicle number, owner name, and unique parking ID.
- Organization and management of parking vehicle and slot records using hash table (`unordered_map`) and vector data structures.
- Automated checking of parking slot availability and selection of the most suitable available slot based on priority.
- Priority-based parking slot allocation using a priority queue, where slots closer to the entrance are given higher priority.
- Management of parking slot status, including Available, Occupied, and Reserved states, along with vehicle-to-slot mapping.
- A console-based parking management interface for vehicle registration, slot allocation, vehicle search, slot status display, and slot release, implemented and tested using Visual Studio Code with a C17-compatible compiler.

Excluded

- Integration with physical IoT parking sensors or camera-based vehicle/occupancy detection.
- Integration with PostgreSQL or other permanent database systems for persistent data storage.
- Machine-learning-based parking demand prediction and predictive analytics.
- Online payment processing, billing, and digital receipt generation.

- Web or mobile application deployment and real-time multi-facility parking management.
- Navigation and shortest-path route computation between parking areas, as these belong to other modules of the wider project.

Assumptions and Boundary Conditions

- Vehicle and parking-slot data used for demonstration are representative sample data and are stored in memory during program execution.
- Each vehicle number is assumed to uniquely identify a vehicle within the prototype.
- Parking slot priority is determined mainly by the distance of the slot from the entrance, with a smaller distance receiving higher priority.
- The initial parking-slot inventory and slot distances are assumed to be predefined before system execution.
- The system assumes that slot status is correctly updated whenever a vehicle is allocated or released.
- The system is developed and evaluated as an academic prototype and is not intended for production deployment without database integration, physical sensor validation, security controls, and further testing.

1.7 Expected Engineering Outcome

The expected outcome is a working C Module 1 prototype that can register a vehicle, generate a unique parking ID, determine the nearest available slot, allocate the slot, display its status, search the registered vehicle, and release the slot. The implementation demonstrates purposeful use of a hash table and priority queue and provides a foundation for integrating later parking-management modules.

CHAPTER 2: LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

The literature and existing-solution material supplied for the project was reviewed to identify common smart-parking approaches, their major benefits, and their limitations. The source material emphasizes parking prediction, IoT-enabled monitoring, occupancy prediction, and AI-supported real-time management. These approaches were considered in relation to the present project, with particular attention to the algorithmic requirements of slot allocation.

2.2 Review of Existing Work

Journal / Source	Author	Year	Key Finding	Limitation
Journal of Traffic & Transportation Engineering	Ma et al.	2024	Improved parking demand prediction	Requires quality data
Transportation Research Procedia	Channamallu et al.	2023	IoT and sensors enable real-time parking	High infrastructure cost
EPJ Data Science	Kaltenbrunner et al.	2025	Predicts parking occupancy	Limited to occupancy prediction
Green Energy & Intelligent Transportation	Channamallu et al.	2026	AI, IoT and real-time monitoring improve parking	Scalability and sensor issues

The detailed project report also cites recent work on smart-city parking prediction, machine-learning-based availability prediction, sensor-enabled parking, route navigation, and AI/IoT parking systems. These works demonstrate the value of prediction and real-time monitoring but also introduce dependencies on data quality, infrastructure, sensors, and scalability. They highlight the importance of selecting suitable data structures and algorithms to ensure efficient

parking-space management and timely decision-making. The reviewed approaches therefore provide useful guidance for developing a reliable and scalable smart parking solution while keeping the current Module 1 implementation focused on registration and slot allocation.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Manual / first-available allocation	Simple to operate	Slow and inconsistent as demand grows	Baseline to improve
Occupancy display	Provides current availability	Does not necessarily assign a specific slot	Module 1 adds allocation logic
IoT / sensor-based parking	Real-time physical occupancy	Infrastructure and maintenance cost	Future integration point
Prediction-based systems	Can anticipate demand	Needs quality historical data	Predictive layer belongs to wider system
Priority-based software allocation	Fast systematic selection	Requires correct data structure maintenance	Core approach for Module 1

2.4 Research / Engineering Gap

The supplied literature focuses strongly on sensing, occupancy prediction, IoT infrastructure, and intelligent transportation. The engineering gap addressed by Module 1 is the software-side foundation for fast, deterministic slot assignment and vehicle tracking. A lightweight prototype can demonstrate efficient registration and allocation without requiring physical sensors or machine-learning infrastructure.

2.5 Proposed Contribution

The proposed contribution is a focused Smart Parking Registration and Slot Allocation module implemented with C data structures. The module combines an `unordered_map` for vehicle lookup and a `priority_queue` for selecting a suitable available slot. It therefore demonstrates how data-structure selection can directly support the operational requirements of a parking facility.

CHAPTER 3: ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

3.1.1 Stakeholder / User Requirements

Stakeholder	Requirement
Driver / Vehicle Owner	Quick registration and suitable slot assignment.
Parking Attendant	Clear current slot status and vehicle details.
Facility Operator	Consistent allocation and release of spaces.
Software Team	Maintainable data structures and predictable operations.

3.1.2 Functional & Non-Functional Requirements

Requirement	Type	Measurable Target
Register vehicle and owner	Functional	Successful registration for valid unique vehicle number
Generate parking ID	Functional	One unique ID per registered vehicle
Check slot availability	Functional	Current status visible before allocation
Allocate suitable slot	Functional	Nearest available slot selected by priority
Search vehicle	Functional	Vehicle record retrieved by vehicle number
Release slot	Functional	Occupied slot returns to Available
Duplicate prevention	Functional	Existing vehicle cannot be registered twice

3.2 Constraints & Applicable Standards

Standard / Guideline	Requirement	How Applied
ISO/IEC 25010	Software quality attributes	Maintainability, reliability and performance considered in module design.
WCAG 2.1	Accessible digital interfaces	Relevant to the future dashboard; console prototype has no graphical accessibility layer.
OWASP guidance	Secure software/API practices	Relevant to future web/API deployment; current prototype has no network API.
Big-O complexity analysis	Document algorithmic efficiency	Core hash and priority-queue operations are analysed.

Only standards and guidelines that are relevant to the supplied project are retained. The current console prototype does not claim compliance with web accessibility or API-security requirements because it does not implement a web interface or network API.

3.3 Design Specifications

Parameter	Target / Requirement	Unit	Priority
Vehicle identifier	Unique vehicle number	text	High
Parking ID	Unique generated ID	text	High
Slot status	Available / Occupied / Reserved	state	High
Slot selection	Lowest distance among free slots	m	High

Vehicle lookup	Hash-based average constant-time lookup	time	High
Allocation	Priority-queue based selection	time	High

3.4 Alternative Solutions & Evaluation

Alternative 1: Linear scan. The system checks every slot from the beginning until it finds a suitable free slot. It is simple but can require $O(n)$ search time.

Alternative 2: Sorted array/vector. Free slots are maintained in an ordered structure. It can provide fast access to the first item but requires additional maintenance when the state changes.

Alternative 3: Priority queue. Free slots are maintained according to a priority such as distance from the entrance. The top item directly represents the highest-priority available slot.

Criterion	Weight	Linear Scan	Sorted Structure	Priority Queue
Performance	High	Medium	High	High
Implementation simplicity	Medium	High	Medium	High
Dynamic allocation suitability	High	Medium	Medium	High
Maintainability	Medium	High	Medium	High
Module 1 relevance	High	Baseline	Good	Best fit

3.5 Selected Design & Detailed Design

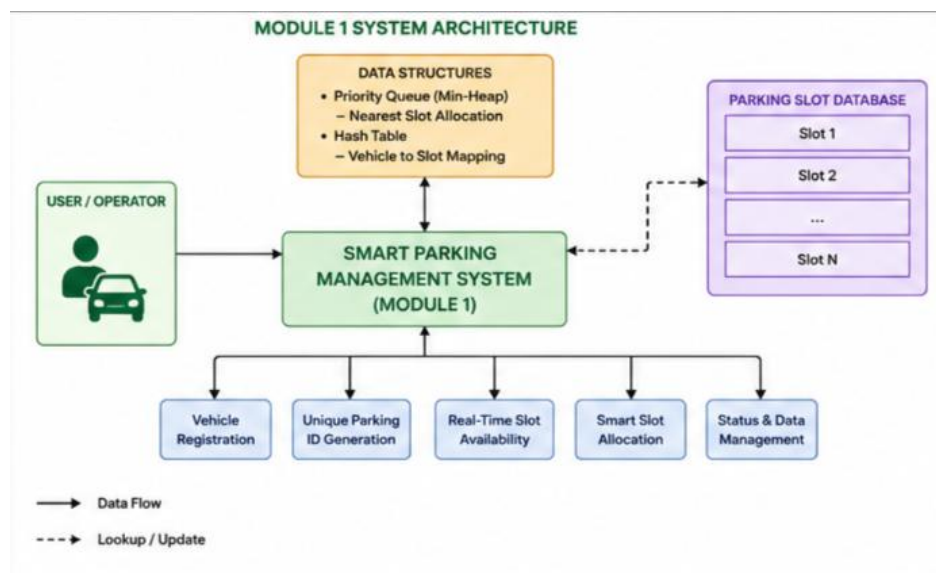
The priority-queue design is selected for smart slot allocation because the project requirement is not merely to find any free slot but to select a suitable slot according to a defined priority. In this prototype, distance from the entrance is the priority. A hash table is selected for vehicle records because the primary lookup key is the vehicle number.

The system is organized into three logical layers: data representation, allocation logic, and user interaction. The data representation layer stores Vehicle and Slot records. The allocation layer

performs registration, allocation, release, and lookup. The user interaction layer provides a menu-based console interface in VS Code.

Core operation flow: Register Vehicle → Generate Parking ID → Store Vehicle Record → Check Available Slots → Select Highest-Priority Free Slot → Mark Slot Occupied → Store Vehicle-to-Slot Mapping. At exit: Find Vehicle → Read Assigned Slot → Mark Slot Available → Clear Vehicle Assignment → Rebuild/refresh Available-Slot Priority Queue.

Key engineering calculation: if n is the number of slots, a priority queue supports insertion and removal of the highest-priority item in $O(\log n)$, while accessing the top item is $O(1)$. An unordered map provides average $O(1)$ lookup for vehicle records. The prototype therefore avoids a complete $O(n)$ vehicle search for every operation.



The four modules of the Smart Parking Management and Predictive Space Allocation System work together through a structured parking-management data layer rather than operating as completely independent components. Module 1 (Smart Parking Registration and Slot Allocation) creates and maintains vehicle and parking-slot records. Module 2 uses parking-area and vehicle information for navigation and route computation, while Module 3 manages parking-related records such as billing, payment details, and parking history. Module 4 uses the available parking information together with predictive analysis to support intelligent parking guidance and demand-based decision-making. This modular dependency allows each component to perform a specific function while contributing to the overall smart parking workflow. Module 1 provides the basic registration and slot-allocation foundation required for the effective operation of the subsequent modules.

Module 1 — Smart Parking Registration and Slot Allocation

- Registers vehicle details, including vehicle number, owner information, and parking requirements.
- Generates a unique parking ID for each registered vehicle.
- Stores and manages vehicle records using a Hash Table (`unordered_map`) for fast searching and retrieval.
- Maintains parking-slot information using suitable data structures such as vectors.
- Uses a Priority Queue to select and allocate the most suitable available parking slot based on defined priority.
- Updates slot status whenever a vehicle is allocated or leaves the parking facility.
- Prevents duplicate vehicle registration and avoids assigning occupied parking slots.

Module 2 — Parking Navigation and Route Management

- Represents the parking facility and its connected locations using a graph data structure.
- Maintains information about parking zones, entrances, exits, and different parking locations.
- Uses weighted edges to represent the distance or cost between different locations.
- Applies shortest-path techniques to determine an efficient route from the entrance to the allocated parking slot.
- Helps reduce unnecessary movement and searching within the parking facility.
- Provides route information that can assist drivers in reaching their assigned parking space efficiently.

Module 3 — Predictive Parking Guidance and Intelligent Management

- Monitors parking-slot availability and occupancy information generated by the parking management system.
- Analyses parking usage information to identify patterns in parking demand and space utilization.

- Uses available parking data to support intelligent and predictive parking-space guidance.
- Helps identify suitable parking options based on current availability and previously recorded parking information.
- Supports improved utilization of available parking spaces by providing systematic parking recommendations.
- Provides a foundation for future integration of advanced predictive analytics, real-time sensors, and intelligent parking-management features.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Vehicle lookup	Linear vector search	Simple	O(n) lookup	unordered_map because vehicle number is the natural key
Slot selection	Linear scan	Very simple	O(n) search	priority_queue because nearest/highest-priority slot is needed
Slot storage	Linked list	Dynamic nodes	Poor indexed access	vector because slot records are compact and easy to display
Persistence	PostgreSQL	Permanent storage	Adds setup/dependency	Excluded from Module 1 prototype; future integration

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Reduce unnecessary internal vehicle searching	Module objective and smart allocation concept	Priority-based allocation selected
Ethics	Avoid unfair or arbitrary assignment	Explicit allocation rule	Deterministic slot priority used

Health & Safety	Avoid directing vehicles to unavailable slots	Slot status validation	Occupied slots are excluded from allocation
Security	Protect vehicle records in future deployment	Hash-based internal records; no network layer	Network/API security deferred to future implementation
Risk	Inconsistent slot and vehicle status	Allocation/release state transitions	Centralized update logic used

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Duplicate vehicle registration	Medium	Medium	Medium	Check hash table before insertion	Low
No free slot	Medium	High	High	Check priority queue before allocation	Low
Incorrect status update	Medium	High	High	Update vehicle and slot records together	Low
Invalid user input	Medium	Medium	Medium	Input validation and recovery	Low
Data loss after program exit	High	Medium	High	Add database persistence in future	Medium

CHAPTER 4: IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The Module 1 implementation follows an iterative, algorithm-first approach. The required operations were first identified from the supplied project documents, followed by selection of data structures that directly support those operations. The implementation was then organized into a single C class so that registration, allocation, search, release, and slot-status operations could be tested independently. Visual Studio Code is used as the development environment and a C 17-compatible compiler is used to build the program.

Resource	Purpose
Visual Studio Code	Source-code editing, terminal execution and debugging
C17 compiler (MinGW g++ or equivalent)	Compilation and execution
C STL unordered map	Hash-table vehicle lookup
C STL priority queue	Priority-based free-slot allocation
C STL vector	Slot record storage
Git / GitHub (optional)	Version control and collaboration

4.2 Software Implementation and Modern Tools Used

The implementation is intentionally focused on Module 1. The Vehicle structure stores vehicle number, owner name, parking ID, and assigned slot. The Slot structure stores slot number, distance from the entrance, current status, and the vehicle using the slot. The SmartParkingModule1 class manages these records and exposes registration, allocation, release, search, and display functions.

The allocation priority is defined using a comparator. A slot with a smaller distance from the entrance receives higher priority. If two slots have equal distance, the lower slot number is selected. This makes the selection deterministic and easy to test.

The supplied earlier project report listed Python/Flask, PostgreSQL, Pandas/NumPy, React, D3.js and Git/GitHub for the wider system. The present Module 1 implementation deliberately

uses C in VS Code because the requested implementation is for the Data Structures course and the module can be demonstrated directly with C standard-library data structures. The broader technologies remain future/integration options rather than claims about this particular prototype.

Tool / Software	Purpose	Stage Used
VS Code	Coding and execution	Implementation / Testing
C17	Module 1 programming language	Implementation
unordered_map	Vehicle record lookup	Registration / Search
priority_queue	Nearest available slot selection	Allocation
vector	Slot state storage	Data management
Git / GitHub	Source versioning	Project management

SmartPark AI

CSA0311 – Data Structures

Dashboard

Vehicle Registration

Smart Slot Allocation

Parking Map & Navigation

Waiting Queue

Vehicle Search

Billing

Parking History

Predictive Analytics

Algorithm Comparison

Data Structures Visualizer

Admin / Simulation

Project Info

Vehicle Registration

Parking ID P1014 – Slot C5 allocated (score 79/100)

Register New Vehicle

Vehicle Number

TN09AB1234

Owner Name

Owner name

Vehicle Type

Car

Special Requirement (optional)

None

Register & Auto-Allocate

Last Registered Vehicle

Vehicle Number

TN76AP5437

Parking ID

P1014

Allocated Slot

C5 (Regular)

Smart Score

79/100

✓ Shortest estimated travel time

✓ Low congestion

✓ Ideal match for vehicle type

✓ Good predicted future availability

4.3 Test Plan & Verification

Requirement	Test Method	Acceptance Criterion
Vehicle registration	Register a new vehicle	Vehicle is stored and parking ID is displayed
Duplicate prevention	Register same vehicle twice	Second registration is rejected
Unique ID	Register multiple vehicles	Each successful registration has a different parking ID
Availability check	Display initial slot status	Free slots are shown as Available
Smart allocation	Register vehicle and allocate	Nearest available slot is selected
Occupied exclusion	Allocate another vehicle	Already occupied slot is not selected
Vehicle search	Search registered vehicle	Correct owner, ID and slot are displayed
Release	Release allocated vehicle	Slot changes from Occupied to Available
No free slot	Fill all slots and allocate another vehicle	System reports no available slot
Invalid vehicle	Allocate/search unregistered vehicle	System rejects or reports vehicle not found

4.4 Results

The report should present actual screenshots and measured timings after the program is executed in VS Code. No fabricated execution measurements are claimed here. The functional acceptance results should be recorded from the implementation using the test plan above.

Test Case	Expected Result	Actual Result	Status
TC01 – New registration	Vehicle registered with unique ID	Fill after execution	Pending
TC02 – Duplicate registration	Duplicate rejected	Fill after execution	Pending

TC03 – Nearest slot allocation	Highest-priority available slot selected	Fill after execution	Pending
TC04 – Occupied slot exclusion	Occupied slot skipped	Fill after execution	Pending
TC05 – Vehicle search	Correct record displayed	Fill after execution	Pending
TC06 – Slot release	Slot becomes Available	Fill after execution	Pending
TC07 – No availability	Allocation rejected	Fill after execution	Pending

Smart Slot Allocation

2/9/2026, 5:35:09 pm

Data Structure Used: Priority Queue (Binary Max-Heap) + Array

Live Slot Score Preview
Preview how the Smart Score + Priority Queue would rank currently available slots for a chosen vehicle type. (Preview only — does not allocate.)

Vehicle Type
Car

RANK	SLOT	TYPE	DISTANCE	CONGESTION	SCORE
1	C2	Regular	39 m	41%	77/100
2	B6	Regular	24 m	38%	73/100
3	D6	Regular	66 m	27%	69/100
4	D2	Regular	40 m	31%	68/100
5	B1	VIP	79 m	24%	67/100
6	B5	Regular	47 m	77%	67/100
7	C4	Regular	28 m	21%	66/100
8	D3	Regular	101 m	37%	63/100

Priority Queue selects: C2 (score 77)

Parking History

2/9/2026, 5:36:27 pm

Data Structure Used: Singly Linked List — insert-at-head $O(1)$, traverse $O(n)$

Linked List Chain
[P1002] → [P1005] → [P1008] → NULL

History Records

PARKING ID	VEHICLE	SLOT	ENTRY	EXIT	DURATION	AMOUNT	
P1002	TN89AB6036	A3	03:38:51 pm	05:36:23 pm	1.96 hr	₹40	Remove
P1005	TN50EF9979	A6	02:55:51 pm	05:36:22 pm	2.68 hr	₹60	Remove
P1008	TN77EF4777	C1	04:36:51 pm	05:36:20 pm	0.99 hr	₹20	Remove

For the wider system, the supplied project report states benchmark results of approximately 12 ms average slot allocation for 1,000 slots, 1.2 ms average vehicle-to-slot lookup, and 38 ms average route computation. Those values relate to the broader reported system and should not be copied as Module 1 C measurements unless independently reproduced

4.5 Data Analysis & Engineering Judgment

The key engineering judgement in Module 1 is the selection of data structures according to the most frequent operations. Vehicle registration and search are keyed by vehicle number, making a hash table appropriate. Slot allocation requires selecting the best free slot, making a priority queue appropriate. This separation of responsibilities avoids forcing one structure to perform every operation.

The prototype also demonstrates an important consistency requirement: allocation must update both the slot record and the vehicle record. Similarly, release must clear the vehicle's assigned slot and return the physical slot to the available set. This is a small-scale example of state consistency that becomes increasingly important in the complete parking-management system.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
Design vehicle/slot data model	Vehicle and Slot structures implemented	Fully
Implement vehicle registration	registerVehicle()	Fully
Generate unique parking ID	generateParkingId()	Fully
Use priority-based smart allocation	priority_queue and comparator	Fully
Provide fast vehicle lookup	unordered_map	Fully
Update slot status	Allocation/release operations	Fully
Test module	Test plan prepared; execution evidence to be added	Partially until screenshots/results are recorded
Integrate sensors/prediction	Outside Module 1 scope	Not applicable to Module 1

4.7 Strengths & Limitations

Strengths:

- Simple and focused Module 1 architecture.
- Fast average vehicle lookup through `unordered_map`.
- Priority-based selection of the nearest available slot.
- Clear slot state transitions.
- Easy to compile and demonstrate in VS Code.
- Provides a foundation for later parking modules.

Limitations:

- Data is stored only in memory and is lost when the program closes.
- No physical sensor or camera input is connected.
- No web dashboard is included in the Module 1 prototype.
- Predictive analytics is not implemented in this module.
- User authentication and network security are not implemented.

The current priority is based on distance only; real facilities may require accessibility, vehicle type, reservation and operational rules.

4.8 Project Planning, Roles & Budget

PHASE	ACTIVITY
Phase 1	Problem identification
Phase 2	Requirement analysis
Phase 3	Literature review
Phase 4	System design
Phase 5	Search Tree implementation
Phase 6	Module development
Phase 7	Eligibility evaluation engine development
Phase 8	Recommendation system development

Phase 9	GUI integration
Phase 10	Testing and debugging
Phase 11	Results analysis
Phase 12	Report preparation

If exact start and end dates for each phase are required, they should be inserted based on the actual project schedule; no dates are fabricated here.

Team Roles

Student	Assigned Responsibility	Actual Contribution
R. Ranjitha	Introduction, Module 1	Introduction, Smart Parking Registration and Slot Allocation
N. Umasri	Objectives, Module 3, Future Scope	Objectives, Predictive Parking Guidance and Intelligent Management, and Future Scope
B. Kiruthika	Problem Statement, Module 2, Conclusion	Problem definition, Parking Navigation and Route Management, and Conclusion

Individual Contribution Percentage

Student	Contribution
R. Ranjitha	35%
N. Umasri	25%
B. Kiruthika	35%
Total	100%

CHAPTER 5: CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

Module 1 establishes the operational foundation of the Smart Parking Management and Predictive Space Allocation System. It addresses vehicle registration, unique parking identification, real-time slot-state management, smart slot allocation, vehicle search, and slot release. The C17 prototype demonstrates how a hash table and priority queue can be selected according to the required operations rather than using a single generic structure for all tasks.

The implementation is suitable for demonstration in Visual Studio Code and provides a clear base for integration with the broader project. The module should be validated with the test cases defined in Chapter 4 and supported with screenshots from the actual execution before final submission.

5.2 Future Work

- Connect the module to PostgreSQL or another persistent database.
- Integrate IoT sensors or camera-based occupancy detection.
- Add the navigation module using a weighted graph and shortest-path algorithm.
- Add predictive arrival and parking-demand analysis.
- Add billing, digital receipts and parking-history management.
- Develop a React/web dashboard for operators and drivers.
- Add authentication, authorization, logging and stronger security controls.
- Extend the allocation rules to support reserved, accessible, electric-vehicle and priority spaces.
- Evaluate performance using larger simulated facilities and real-world data.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired:

- Practical use of `unordered_map` as a hash-table structure.
- Priority-queue based resource allocation.
- Designing data structures around operation requirements.
- State consistency between vehicle and parking-slot records.
- Compiling, testing and debugging a C project in VS Code.

Engineering & Professional Skills Developed:

- Problem decomposition and algorithm selection.
- Complexity analysis using Big-O notation.
- Modular software design.
- Functional testing and debugging.
- Technical documentation and presentation.

Individual Reflection — Ranjitha R

The most difficult engineering problem I encountered was designing the vehicle registration and parking-slot allocation process so that vehicle information could be stored efficiently and suitable parking spaces could be allocated without conflicts. I addressed this by using a Hash Table (`unordered_map`) for storing and quickly retrieving vehicle records and a Priority Queue for organizing available parking slots according to their priority. A key engineering decision I made was to give higher priority to suitable slots based on the defined parking allocation criteria, allowing the system to make a consistent decision instead of relying on arbitrary manual selection. Through this work, I gained a better understanding of how data structures such as hash tables and priority queues can be applied to a practical resource-allocation problem. If I were to redesign this module, I would add stronger input validation, persistent database storage, and real-time parking-sensor integration to improve reliability and scalability.

Individual Reflection — Kiruthika B

The most difficult engineering problem I faced was designing an efficient navigation mechanism that could represent the connections between different parking locations and help determine a suitable route. I addressed this by representing the parking facility using a graph structure, where locations are treated as nodes and the connections between them are represented as weighted edges. A key engineering decision was to use distance-based weights so that the navigation process could consider the actual cost or distance between parking locations when determining a route. This work improved my understanding of how graph-based data structures and shortest-path concepts can be applied to real-world parking and navigation problems. If I were to redesign this module, I would include dynamic route updates based on parking congestion and integrate real-time information about blocked or unavailable routes.

Individual Reflection — Uma

The most difficult engineering problem I encountered was developing the predictive parking guidance component so that available parking information could be used to support better parking decisions. I addressed this by analysing parking availability and usage information and connecting it with the intelligent management functions of the overall system. A key engineering decision was to focus on systematic analysis of parking information rather than depending only on manually selected parking choices, as this provides a better foundation for intelligent space allocation and future prediction. Through this work, I gained a clearer understanding of how data generated by parking-management modules can be used to support predictive analysis and intelligent guidance. If I were to redesign this module, I would incorporate a larger historical dataset, real-time sensor information, and more advanced predictive techniques to improve the accuracy of parking-demand estimation.

REFERENCES

- [1] X. Xiao, Z. Peng, Y. Lin, Z. Jin, W. Shao, R. Chen, N. Cheng, and G. Mao, “Parking Prediction in Smart Cities: A Survey,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 24, no. 10, pp. 10302–10326, 2023.
- [2] N. Nooripour, “Smart Parking Systems: Comprehensive Review Based on Technological Perspective,” in *Proc. IEEE*, 2024.
- [3] A. Dahiya, P. Mittal, Y. K. Sharma, U. K. Lilhore, S. Simaiya, E. Ghith, and M. Tlija, “Machine Learning-Based Prediction of Parking Space Availability in IoT-Enabled Smart Parking Management Systems,” *Journal of Advanced Transportation*, 2024.
- [4] C. Ma et al., “A Review of Research on Urban Parking Prediction,” *Journal of Traffic and Transportation Engineering (English Edition)*, vol. 11, no. 4, pp. 700–720, 2024.
- [5] G. Sekar, P. M. Benson Mansingh, J. Prasad, and A. Britto Manoj, “Sensor Enabled Vehicle Parking System Using Blynk IoT Application,” in *Proc. 2024 5th Int. Conf. Smart Electronics and Communication (ICOSEC)*, 2024.
- [6] D. Bhatnagar, V. Soni, and P. Agrawal, “IoT-Enabled Smart Car Parking System with Route Navigation,” in *Proc. 2024 IEEE 1st Int. Conf. Green Industrial Electronics and Sustainable Technologies (GUEST)*, 2024.
- [7] M. R. Rashidi, C. Cherian, V. S. Negi, and S. Chinara, “IoT-Enabled Smart Parking System Using Artificial Intelligence and Optical Character Recognition,” in *Proc. 2024 6th Int. Conf. Communications, Signal Processing, and their Applications (ICCSPA)*, pp. 1–6, 2024.
- [8] J. W. Simatupang and Y. S. Alfarizi, “Smart Parking System with IoT-Enabled Using NodeMCU for Car Reservation in Public Areas,” in *Proc. 2024 IEEE 2nd Int. Conf. Electrical Engineering, Computer and Information Technology (ICEECIT)*, pp. 107–112, 2024.
- [9] H. Kuang, K. Deng, Q. Wang, W. Ye, H. Qu, and J. Li, “Deep Meta-Learning Approach for Regional Parking Occupancy Prediction Considering Heterogeneous and Real-Time Information,” *Advanced Engineering Informatics*, vol. 64, Art. no. 102969, 2025.
- [10] S. S. Channamallu, S. Kermanshachi, J. M. Rosenberger, and A. Pamidimukkala, “Smart Parking Systems: A Comprehensive Review of Digitalization of Parking Services,” *Green Energy and Intelligent Transportation*, vol. 5, no. 1, Art. no. 100293, 2026.

- [11] IEEE, “Smart Parking Management Using IoT and Artificial Intelligence,” IEEE Xplore Digital Library, 2025.
- [12] Springer Nature, “Predictive Parking Space Allocation Using Machine Learning Techniques,” Springer, 2025.
- [13] IJCRT, “Advanced Data Structure-Based Smart Parking Management System,” International Journal of Creative Research Thoughts, 2025.
- [14] IJRASET, “Real-Time Smart Parking System with Predictive Analytics,” International Journal for Research in Applied Science & Engineering Technology, 2026.
- [15] D. Sahu, P. Sinha, S. Prakash, T. Yang, R. S. Rathore, and L. Wang, “A multi objective optimization framework for smart parking using digital twin pareto front MDP and PSO for smart cities,” Scientific Reports, vol. 15, Art. no. 7783, Mar. 2025.
- [16] ISO/IEC 25010, Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model.
- [17] W3C, Web Content Accessibility Guidelines (WCAG) 2.1.
- [18] OWASP Foundation, Application Security Verification and secure software development guidance.
- [19] C Standard Library documentation for `unordered_map`, `priority_queue`, `vector` and related containers.

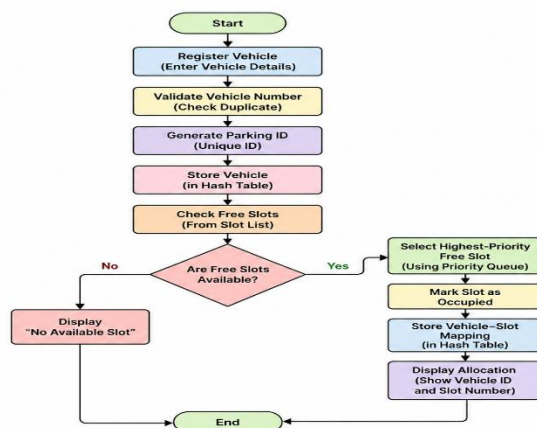
APPENDICES

Appendix A – Detailed Calculations and Complexity

Operation	Primary Structure	Average / Typical Complexity
Register vehicle	unordered_map	O(1) average
Search vehicle	unordered_map	O(1) average
Select best available slot	priority_queue	O(1) top access
Remove selected slot	priority_queue	O(log n)
Insert/rebuild available slots	priority_queue	O(log n) per insertion
Display all slots	vector	O(n)
Release and refresh slots	vector + priority_queue	O(n) in current prototype due to refresh

The current prototype rebuilds the available-slot priority queue after allocation/release for simplicity and consistency. A production implementation could maintain the heap incrementally and use lazy deletion or indexed-heap techniques to reduce unnecessary rebuilding.

Appendix B – Engineering Flowchart



Appendix C – Source Code / Code Repository Information

The complete Module 1 C source code is supplied separately with this report in the Smart_Parking_Module_1_VSCode folder. The main file is main.cpp. It can be opened directly in Visual Studio Code.

Appendix D — Datasheets

Not applicable to this project, as no hardware components were used.

Appendix E — Experimental / Raw Data

Not applicable to this project.

Appendix F — Ethics / Institutional Approval

Not applicable, as the project used illustrative/sample data rather than actual citizen data requiring institutional ethics approval.

Appendix G – Project Meeting / Progress Records

Review 1 on 27/7/2026

During the first review, we explained and presented our proposed project topic, its objectives, and the overall concept of the system.

The review panel suggested making necessary changes to the system architecture to improve the clarity and functionality of the proposed system.

Based on the feedback, we revised the architecture and proceeded with the implementation of the system.

Review 2 on 28/8/2026

During the second review, we presented the changes made to the system architecture based on the feedback received during the first review.

We also demonstrated the implementation progress and working modules of the system. The implemented features and overall system functionality were explained to the review panel.

Appendix H – User / Industry Feedback

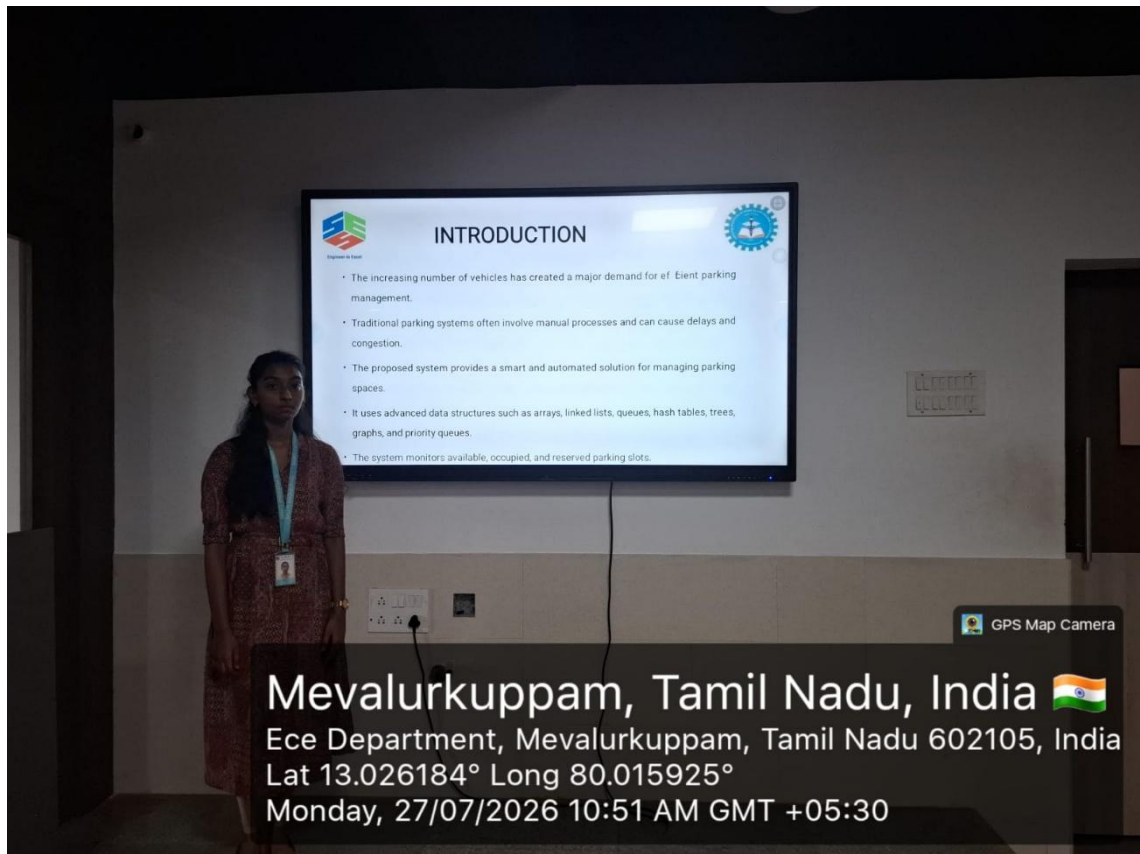
Not applicable / not collected for this prototype.

Appendix I – Publications / Patents / Competitions / Product Outcomes

None report.

Appendix J – Individual Contribution Evidence

Review Photos:



CAPSTONE PROJECT OUTCOME MAPPING SHEET

Outcome	Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	Problem statement and background analysis	SO1	Ch. 1, Ch. 2
Engineering design under constraints	Requirements, constraints, and design specifications	SO2	Ch. 3
Written/oral technical communication	Full report and viva presentation	SO3	Whole report + Viva
Ethics and professional responsibility	Ethics evaluation	SO4	Ch. 3 (3.7)
Teamwork and leadership	Team roles and contribution table	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	Test plan, results, and analysis	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	Individual reflections	SO7	Ch. 5 (5.3)
Standards	Constraints and applicable standards table	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	Sustainability evaluation	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	Risk register	SO2/SO4	Ch. 3 (3.7)
Security considerations	Security evaluation	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	Selected design and module interaction	SO1/SO2	Ch. 3 (3.5)
Project planning and management	Timeline and roles table	SO5	Ch. 4 (4.8)
Individual contribution	Contribution statement and Ch. 4 (4.8)	SO1–SO7	Contribution Statement + Ch. 4 (4.8)